

Project Proposal - Plants vs. Zombies like game

Jackson Selfridge, Gonzalo Domingo, David Filipovas

1. Introduction

- a. Plants vs. Zombies is a small 2D game that can be created in Godot. The player places plants to stop oncoming waves of zombies
- b. Scope – this game will contain one level, with every wave defeated increasing the number of zombies and higher strength zombies until the player becomes overwhelmed. It will include:
 - i. A select amount of plants to place to defend
 - ii. Randomly generated waves of zombies that spawn
 - iii. Different lanes for plants and zombies to be
 - iv. Collision detection of the projectiles
 - v. Highest amount of waves survived as a goal
 - vi. Lose condition (zombies get to the end)
 - vii. A simple user interface
 - viii. Resource management
 - ix. The project will not include online multiplayer or full-saved games functionality.

Users – this game will be intended to play as a click and drag or click and place game

2. - Overview – this game will start with the player having a limited amount of sun to start planting certain plants to grow their defense. The zombies start approaching after a bit. The game ends when the zombies reach the house.

Controls – The player will click on the plants, selecting one, then placing one if they have enough resources to. Esc will pause the game.

3. Functional Requirements

FR-1: Plant Selection and Placement

The system shall allow the player to select a plant and place it in a valid location using the mouse.

FR-2: Collision

The game shall detect collisions between projectiles and zombies, causing zombies to take damage. Zombies shall also attack plants when they are within range.

FR-3: Plant Health

The player has no health, just the zombies reaching the door. The plants will have their own set health though.

FR-4: Game-over condition

When a zombie reaches the front door, the game ends.

FR-5: Enemy Movement/behavior

Enemies shall automatically move through the lawn. Zombies will move towards the end goal, destroying any plants in their way. Upon completion of the wave, harder zombies will be introduced and more zombies will appear (a multiplier of some kind to spawn zombies)

FR-6: Resource Management

When the player clicks on a sun, it shall be collected and added to the player's resource bank.

FR-7: Shop Feature

Every two waves, the player shall be able to choose one new plant from three available options.

FR-8: High Score and Difficulty Progression

The game shall save the highest number of waves completed by the player. Each successive wave shall increase in difficulty.

FR-9: Restarting

The game-over screen and pause menu shall allow the player to restart the game.

FR-10: Pause

The player shall be able to pause and resume the game.

4. Non-Functional Requirements

NFR-1: Performance

The game should maintain approximately 60 frames per second on the computers available in the classroom.

NFR-2: Usability

A new player should be able to determine the game's controls without external instructions.

NFR-3: Maintainability

Game functionality should be divided among multiple Godot scenes and scripts rather than implemented in a single script.

NFR-4: Version Control

The project shall be maintained in a Git repository.

NFR-5: Platform

The game shall run on the classroom computers using the version of Godot specified for the course.

NFR-6: Sounds

The game will have hit sounds and collect sounds for when things are collected

5. Major Game Components

The anticipated major components are:

- Selected number of plant types
- Enemies
- Bank system
- User interface
- Main menu
- Ramping up gameplay
- Game-over screen

These components may correspond to separate Godot scenes, scripts, or systems.

6. Use Cases

Use Case 1: Sun collection

****Actor:**** Player

****Precondition:**** The sun has spawned but not collected

****Normal Flow:****

1. The player moves the mouse toward the sun sprite.
2. The player clicks on the sun
3. The game removes the sun from the scene
4. The game records that the player has collected the sun in their bank

****Postcondition:**** The player now has more sun in the bank

Use Case 2: Place a Plant

****Actor:**** Player

****Precondition:**** The player has enough sun to purchase the selected plant.

****Normal Flow:****

1. The player moves the mouse to the plant they wish to place
2. The player clicks on the plant
3. The game recognizes that the player has enough sun to plant that plant
4. The game allows the player to place the plant successfully

****Postcondition:**** The plant is placed on the lawn and becomes active.

7. Acceptance Criteria

The project will be considered complete when:

- The player can place plants and collect sun.
- Plants cannot be purchased if the player does not have enough sun.
- The sun balance updates correctly when sun is collected or spent.
- The game ends when a zombie reaches the player's house.
- Zombies can kill the plants.
- Projectiles correctly collide with and damage zombies.
- Zombie waves become progressively more difficult.
- The number of completed waves is tracked as the player's score.
- The game-over screen allows the player to restart.
- The player can pause and resume the game.
- The game runs successfully from a clean checkout of the Git repository.

###8. Risks/Technical Challenges

- Zombies overlapping or getting stuck
 - o Using collision detection to make sure this doesn't happen
- Game runs too slow
 - o Remove the zombies off the screen when they die
- Game becomes too difficult
 - o Slowly increase the difficulty and strength of each zombie after each wave

9. Extra Requirements

- Plant removal
- More plant types
- More zombie types
- Music
- Local two-player functionality
- Different stages